

Ex. No. 06	INTEGRATE ANIMATIONS AND TRANSITION
Date of Exercise	
Git Hub Host Link	https://github.com/Karthikeyan-0/flutter.git

Aim

To implement animations and page transitions in Flutter.

Description

This experiment demonstrates the use of Flutter's `AnimatedContainer` widget to create a smooth size animation. A Boolean state variable `isLarge` is used to control the size of the container. When the Animate button is pressed, `setState()` changes the value of `isLarge`, and the `AnimatedContainer` smoothly changes its width and height.

Procedure:

1. A new Flutter project named `animation_app` was created using the Flutter command.
2. The project was opened in Visual Studio Code.
3. A `StatefulWidget` was created to maintain the animation state.
4. A Boolean variable `isLarge` was initialized as `false` to control the size of the animated container.
5. An `AnimatedContainer` widget was added with a duration of one second.
6. The width and height of the container were set based on the value of `isLarge`. The container changes from 150×150 to 300×300 .
7. An `ElevatedButton` named Animate was added to control the animation.
8. When the button is pressed, `setState()` changes the value of `isLarge`.
9. The application was executed and the Animate button was clicked to observe the smooth size transition.

Program

```
import 'package:flutter/material.dart';

void main() {

  runApp(const AnimationApp());

}

class AnimationApp extends StatelessWidget {

  const AnimationApp({super.key});
```

@override

```
Widget build(BuildContext context) {
```

```
  return MaterialApp(
```

```
    debugShowCheckedModeBanner: false,
```

```
    home: const AnimationPage(),
```

```
  );
```

```
}
```

```
}
```

```
class AnimationPage extends StatefulWidget {
```

```
  const AnimationPage({super.key});
```

@override

```
  State<AnimationPage> createState() => _AnimationPageState();
```

```
}
```

```
class _AnimationPageState extends State<AnimationPage> {
```

```
  bool isLarge = false;
```

@override

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    appBar: AppBar(
```

```
      title: const Text('Animation'),
```

```
    ),
```

```
    body: Center(
```

```
      child: Column(
```

```
        mainAxisAlignment: MainAxisAlignment.center,
```

```
children: [  
  AnimatedContainer(  
    duration: const Duration(seconds: 1),  
    width: isLarge ? 300 : 150,  
    height: isLarge ? 300 : 150,  
    decoration: BoxDecoration(  
      borderRadius: BorderRadius.circular(20),  
      color: Colors.blue,  
    ),  
    child: const Center(  
      child: Text(  
        'Flutter',  
        style: TextStyle(  
          color: Colors.white,  
          fontSize: 24,  
        ),  
      ),  
    ),  
  ),  
  const SizedBox(height: 30),  
  ElevatedButton(  
    onPressed: () {  
      setState(() {
```

```
    isLarge = !isLarge;

    });

    },

    child: const Text('Animate'),

  ),

  ],

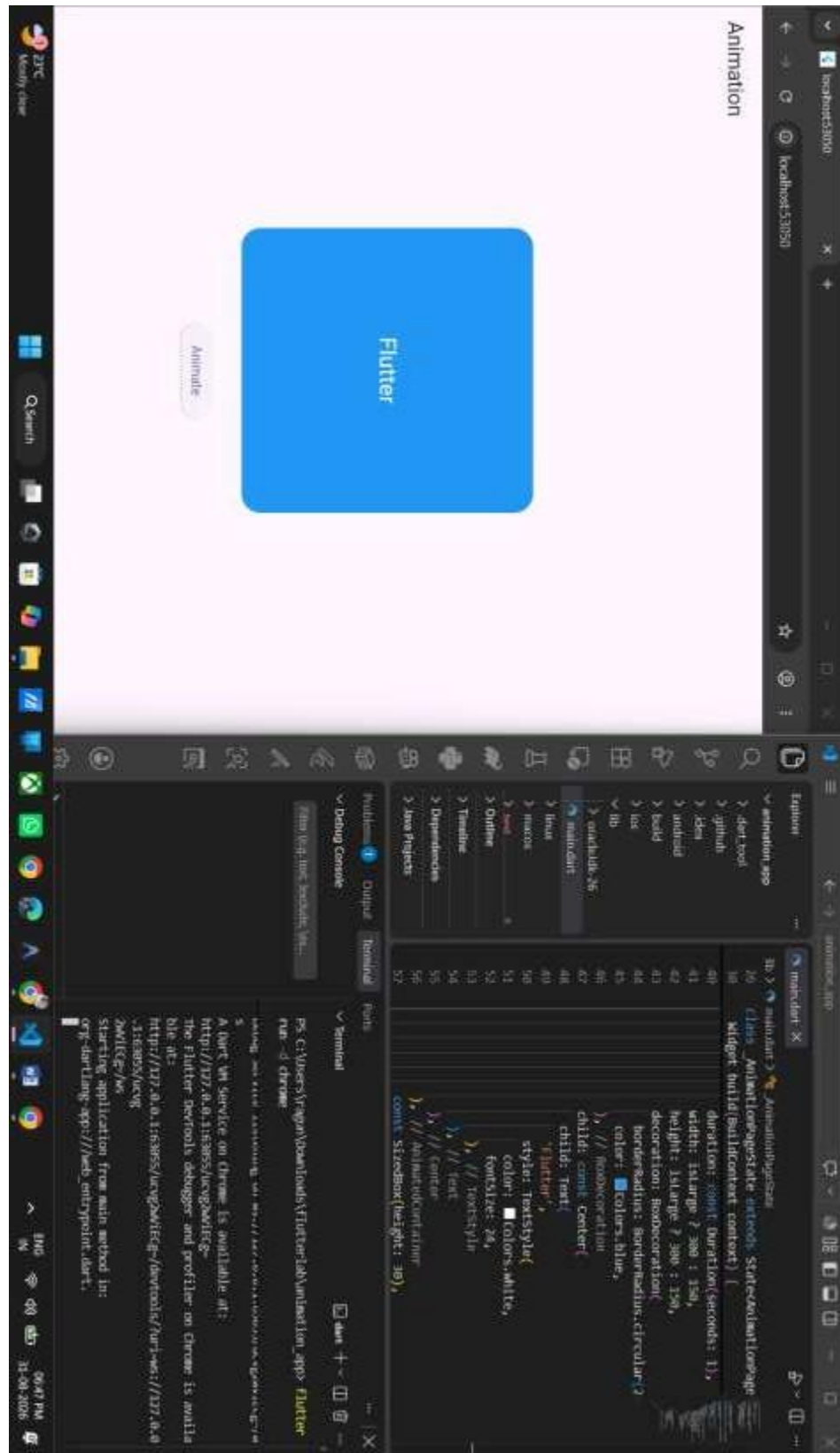
),

),

);

}

}
```



Result

Thus, the Flutter application was successfully developed to demonstrate animation using the `AnimatedContainer` widget. On clicking the Animate button, the container smoothly changes its size, demonstrating Flutter's animation behavior.